INSTITUTO TECNOLÓGICO DE MAZATLÁN

Tecnológico Nacional de México

Reporte Técnico

Práctica 4.1 — Simulador de Oferta y Demanda con Potenciómetros: Arduino UNO R4 WiFi, Make y Telegram

Alumna:	Díaz Camacho Karla Mariel
Matrícula:	251000165
Materia:	Desarrollo Sustentable (ACD-0908)
Profesor:	Barrón Hernández Miguel Ángel
Fecha:	10 de noviembre de 2026

1. Resumen general del proyecto

El proyecto es un simulador físico del mercado del camarón. Dos potenciómetros representan la demanda y la oferta de un kilo de camarón; el Arduino UNO R4 WiFi calcula el precio de equilibrio, lo muestra en su matriz LED y, cuando detecta escasez (la demanda supera a la oferta por un margen definido), envía una alerta a Telegram a través de un webhook de Make. Así, un fenómeno económico se convierte en datos y en una notificación automática.

2. Explicación técnica

2.1 Flujo general

```
Potenciómetros -> Arduino UNO R4 WiFi (calcula y muestra el precio) -> Webhook de Make -> Telegram (aler
```

2.2 Circuito

Componente	Pata izquierda	Pata central	Pata derecha
Potenciómetro de demanda (10 kΩ)	GND	A0	5V
Potenciómetro de oferta (10 kΩ)	GND	A1	5V

Si al girar un potenciómetro a la derecha el valor baja en lugar de subir, se invierten los cables de las patas de los extremos. La placa se conecta por USB-C a la computadora para programarla y para abrir el Monitor Serie a 9600 baudios.

2.3 Modelo económico en el código

Cada potenciómetro se lee con `analogRead()` (0 a 1023) y se convierte a una escala de 10 a 90 con `map()`. Después se usa un modelo lineal sencillo, donde la demanda sigue P = a – Q y la oferta sigue P = Q – b:

```
int demanda = map(analogRead(PIN_DEMANDA), 0, 1023, 10, 90);
int oferta  = map(analogRead(PIN_OFERTA),  0, 1023, 10, 90);

int a = 100 + demanda;
int b = oferta - 50;
int precio = ((a - b) / 2) * 4;   // pesos por kilo
```

Al simplificar, el precio queda como **300 + 2 × (demanda** – **oferta)**: cuando demanda y oferta son iguales el precio de equilibrio es de $300 por kilo; con demanda 80 y oferta 20 el precio es de $420, justo el dato de prueba de la guía. Con los límites de 10 a 90, el precio se mueve entre unos $140 y $460 por kilo.

2.4 Detección de escasez y alerta

```
bool escasez = (demanda - oferta) >= 25;

if (escasez && !enEscasez && millis() - ultimaAlerta > ESPERA_ALERTA) {
  enviarAMake(demanda, oferta, precio);
  ultimaAlerta = millis();
}
enEscasez = escasez;
```

Hay escasez cuando la demanda supera a la oferta por 25 puntos o más (equivale a un precio de $350 o más). La alerta se envía solo cuando el mercado *pasa* a escasez y como máximo una vez por

minuto (`ESPERA_ALERTA = 60000`), por lo que no se gastan créditos de Make en cada lectura. El precio se muestra en la matriz LED con la librería `Arduino_LED_Matrix` y `ArduinoGraphics`.

2.5 Conexión WiFi y envío a Make

El programa se conecta a una red WiFi de 2.4 GHz con usuario y contraseña (no funciona con redes que piden iniciar sesión en una página). Cada equipo debe cambiar cuatro datos: el nombre y la clave de la red, el host del webhook y su ruta. Si la URL del webhook es `https://hook.us2.make.com/abc123`, el host es `hook.us2.make.com` y la ruta es `/abc123`.

La alerta se envía con una petición HTTPS GET (`WiFiSSLClient`, puerto 443) que lleva los datos en la propia dirección:

```
GET /RUTA?demanda=80&oferta=20&precio=420&estado=escasez
```

2.6 Escenario en Make

#	Módulo	Función
1	Webhooks – Custom webhook	Recibe la petición del Arduino y detecta los campos demanda, oferta, precio y estado. Para detectarlos se usa Run once y se abre en el navegador la URL del webhook con los datos de prueba.
2	Telegram Bot – Send a Text Message or a Reply	Envía la alerta al Chat ID del equipo con la conexión del bot Mercado Equipo #.

El texto del mensaje de Telegram usa los datos recibidos:

```
ALERTA DE ESCASEZ
Demanda: {{1.demanda}}
Oferta: {{1.oferta}}
Precio de equilibrio: ${{1.precio}} por kg
La demanda supera a la oferta: el precio sube.
```

El Chat ID se obtiene enviando un mensaje al bot y abriendo `https://api.telegram.org/botTOKEN/getUpdates`, donde se copia el número que sigue a “chat”:{“id”:. El token es la llave del bot, por lo que no debe compartirse ni subirse a GitHub. Por último, el escenario Mercado_Equipo# se guarda y se activa (ON).

3. Resultados esperados

- El Monitor Serie muestra, a 9600 baudios, la demanda, la oferta y el precio en cada lectura, y la matriz LED muestra el precio.
- Con la demanda al máximo y la oferta al mínimo, el Arduino envía una alerta y llega el mensaje de escasez a Telegram.
- Si la escasez continúa, no se repite la alerta; solo se envía de nuevo si el mercado sale de la escasez y vuelve a entrar, con al menos un minuto de diferencia.
- Si la oferta supera a la demanda, el precio baja y no se envía ninguna alerta.
- El historial (History) de Make permite revisar cada alerta recibida.